

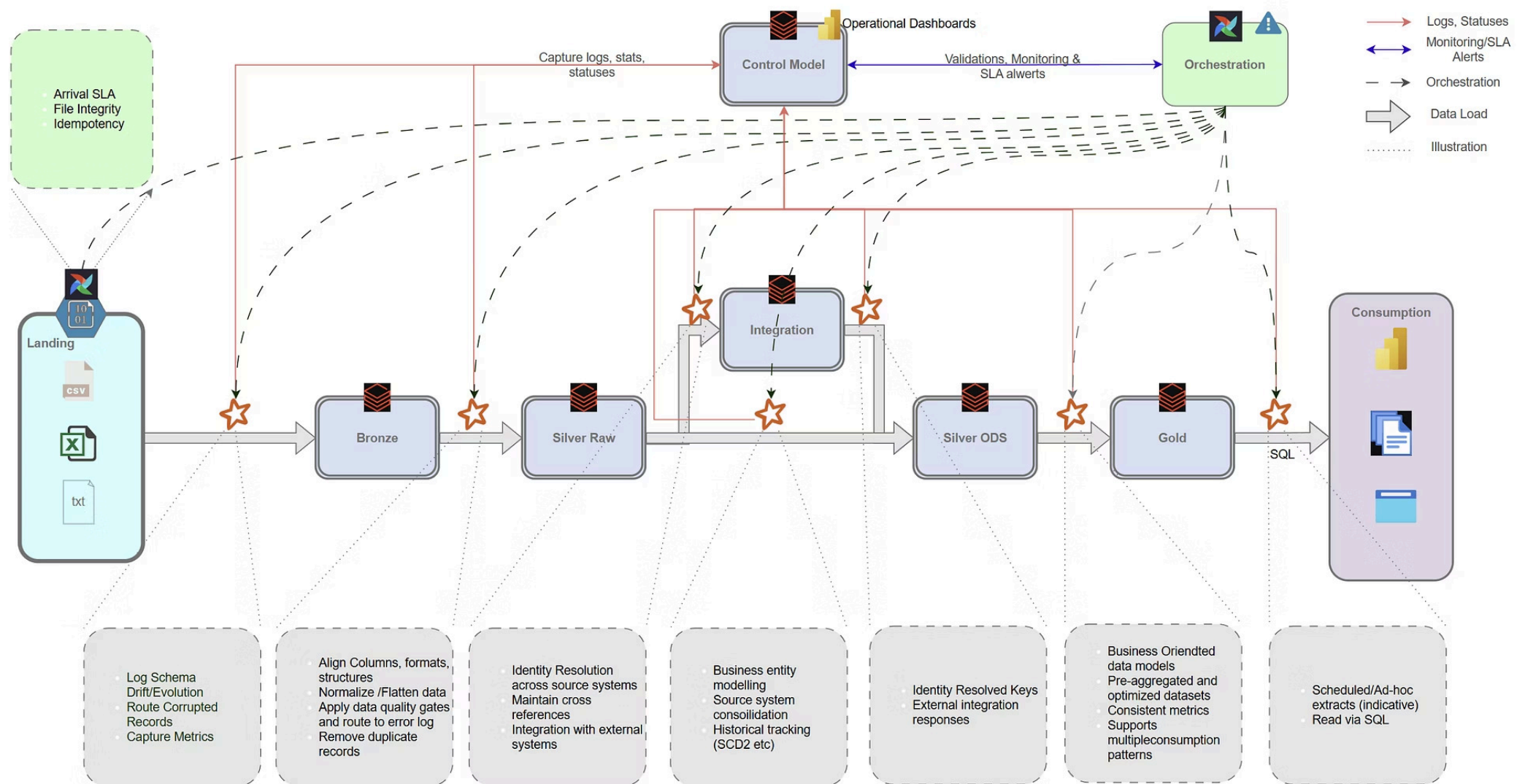
Data Platform Architecture & Control Model

A technical overview of the Medallion Architecture on Databricks + Airflow, purpose-built for healthcare data — Claims, Eligibility, ADT, and Providers — with reliability, observability, and scalability at its core.

[TECHNICAL ARCHITECTURE OVERVIEW](#)[NOT FUNCTIONAL DESIGN](#)

Data Platform Architecture

Medallion Architecture on Databricks + Airflow — from Landing to Consumption, governed by the Control Model



The Iceberg of Data Pipeline Complexity

What stakeholders see is just the surface — the real complexity runs deep.



Above the Surface — Visible to Stakeholders

Late or Missing Reports

Pipeline Failures

Inconsistent Metrics

Below the Surface — Hidden Complexity

Ingestion Gaps — invalid, missing & duplicate files

Schema Drift — evolving structures break downstream

Silent Failures — no execution tracking or audit trail

No SLA Visibility — issues surface only after impact

Idempotency Risks — duplicate processing goes undetected

Control Model – Governing Data Reliability

Every decision is driven by one principle: **Expected vs. Actual**

Ingestion Control

File Validation — Format, size, count & integrity checked at arrival

Arrival Monitoring — Expected vs. actual file timing tracked per feed

Idempotency — Files fingerprinted; re-runs skip already-processed data

Schema Drift Detection — Structure changes detected at ingestion, alerted immediately

Processing Control

Execution Tracking — Every stage tracked: duration, record counts, status

Batch Ordering — Sequential processing enforced; no out-of-order execution

Data Reliability & SLA

Completeness Validation — Record counts reconciled expected vs. actual wherever needed

Data Quality & Error handling
— Invalid records isolated; valid records continue processing

SLA Tracking — Every milestone classified: On-Time / Delayed / Breached

Alerting — Routed to email, Slack, or ticketing based on severity

Stage 1 — Ingestion Controls

Ensuring only valid, expected, non-duplicate data enters the pipeline boundary.

1 · File Validation & Integrity

Problem: Files that are corrupted, wrongly formatted, or incomplete can silently enter the pipeline.

Resolution: Every file is checked for format, size, count and integrity at arrival — rejected immediately if any check fails.

2 · Arrival Monitoring

Problem: Missing or late files are only discovered when downstream reports break — often hours later.

Resolution: Expected vs. actual arrival times are compared continuously — teams are alerted before SLA windows are missed.

3 · Duplicate Prevention

Problem: Pipeline retries cause the same file to be processed twice, creating duplicate records.

Resolution: Each file is uniquely identified by path and name — already-processed files are automatically skipped on re-run.

4 · Batch Ordering & Dependencies

Problem: Out-of-order batch execution corrupts aggregations and time-series calculations.

Resolution: Each batch verifies the prior day's batch completed successfully before it is allowed to proceed.

5 · Schema Drift Detection

Problem: Source system structure changes silently break downstream transformations without warning.

Resolution: Incoming file structure is compared field-by-field against the registered baseline — deviations are logged and alerted without blocking ingestion.

Stage 2 & 3 — Processing & Data Reliability

Tracking every execution and verifying data landed correctly after each write.

6 · Pipeline Execution Tracking

Problem: Diagnosing failures requires hunting across multiple logs, job histories, and cloud storage.

Resolution: Every pipeline stage is tracked centrally — start time, end time, record counts, and status are queryable in real time without touching logs.

7 · Data Completeness Validation

Problem: A pipeline can report success while writing only a fraction of the expected records — partial loads go undetected.

Resolution: Record counts are compared expected vs. actual after every write — any shortfall immediately triggers an alert and marks the stage incomplete.

8 · Error & Data Quality Handling

Problem: A single bad record either blocks the entire batch or silently corrupts the dataset downstream.

Resolution: Invalid records are isolated with a failure reason — valid records continue processing, and every record is accounted for through reconciliation.

Stage 4 — Observability & SLA

Making platform behaviour visible, measurable, and actionable in real time.

9 · SLA Tracking & Centralized Alerting

Problem: No consistent way to measure whether data arrived and processed on time, or to route alerts to the right teams.

Resolution: Every processing milestone is classified as On-Time, Delayed, or Breached — alerts are automatically routed to Slack, email, or ticketing based on severity and feed criticality.

SLA Status Classifications

●●●●● 100%

✓ On-Time

Actual timestamp within expected window

●●●●○ 50%

⚠ Delayed

Within grace period — warning alert sent

○○○○○ 0%

🔴 Breached

Exceeds grace period — high-priority alert sent

Alert Channels

→ Slack — Channel-routed by domain or severity

→ Email — Distribution lists for SLA breaches

→ Ticketing — Auto-ticket creation for critical failures

→ Webhook — Custom integrations supported

📌 All 9 use cases reduce to one question: Does what actually happened match what was expected?